

Chapter 3

Arrow functions

Object-oriented TypeScript, test first

Marks Table

Sort by name

Sort by mark

☒ Passes only

8 students · average 60.9 · top: Emma (81)

Name	Mark	Grade
Emma	81	A
Aoife	72	A
Isla	68	B
Chloe	65	B
Grace	59	C
Darragh	54	C
Finn	47	D
Jack	41	D

Today

- what `() => { ... }` really is
- arrow functions vs Java lambdas vs `function`
- arrows as listeners - and what they **remember**
- the `this` trap
- `map`, `filter`, `reduce`, `find`, `toSorted`
- functions stored in data

Functions are values

- Chapter 2: `addEventListener("click", handleToss)` - the function **itself**
- a value can be stored, put in an array or object, passed, returned
- so can a function
- arrow functions: write a function **anywhere a value can go**, no name needed

Three ways

The same function, three ways

```
function toFahrenheit(celsius: number): number { return celsius * 9 / 5 + 32; }
```

a function declaration (Chapter 1)

```
const toFahrenheit = (celsius: number): number => { return celsius * 9 / 5 + 32; };
```

an arrow function with a { block } body - it needs return

```
const toFahrenheit = (celsius: number): number => celsius * 9 / 5 + 32;
```

an arrow function with an expression body - the value after => is returned

Java lambda: `celsius -> celsius * 9 / 5 + 32`

TypeScript arrow: `(celsius) => celsius * 9 / 5 + 32` => (equals), not -> (minus)

Reading an arrow function

```
export const toFahrenheit = (celsius: number): number =>  
  celsius * 9 / 5 + 32;
```

- `(celsius: number)` - parameters
- `: number` - return type
- `=>` - "gives"
- expression body → **returned**, no `return`
- `{ block }` body → needs `return`

Java lambdas and TypeScript arrows

Java	TypeScript
<code>c -> c * 9 / 5 + 32</code>	<code>(c) => c * 9 / 5 + 32</code>
<code>(a, b) -> a + b</code>	<code>(a, b) => a + b</code>
<code>Function<Double, Double></code>	<code>(c: number) => number</code>
effectively final variables only	any variable it can see

`=>` - an equals sign, not a minus

Tests are arrow functions

```
Deno.test("water boils at 212 °F", () => {  
  assertEquals(toFahrenheit(100), 212);  
});
```

`Deno.test(name, function)` - Deno stores the function and calls it later

Just like `addEventListener`

Decimals are not exact

```
assertEquals(toFahrenheit(36.6), 97.88)
```

```
- 97.880000000000001  
+ 97.88
```

```
assertAlmostEquals(toFahrenheit(36.6), 97.88);
```

0.1 + 0.2 is 0.30000000000000004 - in Java too

Project 2: arrow listeners

```
for (const points of POINTS) {  
  const button = document.createElement("button");  
  button.textContent = `+${points}`;  
  button.addEventListener("click", () => {  
    team.addPoints(points);  
    render();  
  });  
  container.appendChild(button);  
}
```

Six buttons, one loop - each listener made inside it

The scoreboard

Basketball Scoreboard

Home

5

+1

+2

+3

Away

2

+1

+2

+3

Home lead by 3

New game

Closures

- the listener runs **later** - after `addButtons` has finished
- it still knows `team` and `points`
- an arrow function **remembers the variables it could see** when it was made
- function + remembered variables = a **closure**
- each time round the loop: a new function, its own `points`

? . - a shorter null check

```
resetButton?.addEventListener("click", () => {  
  home.reset();  
  away.reset();  
  render();  
});
```

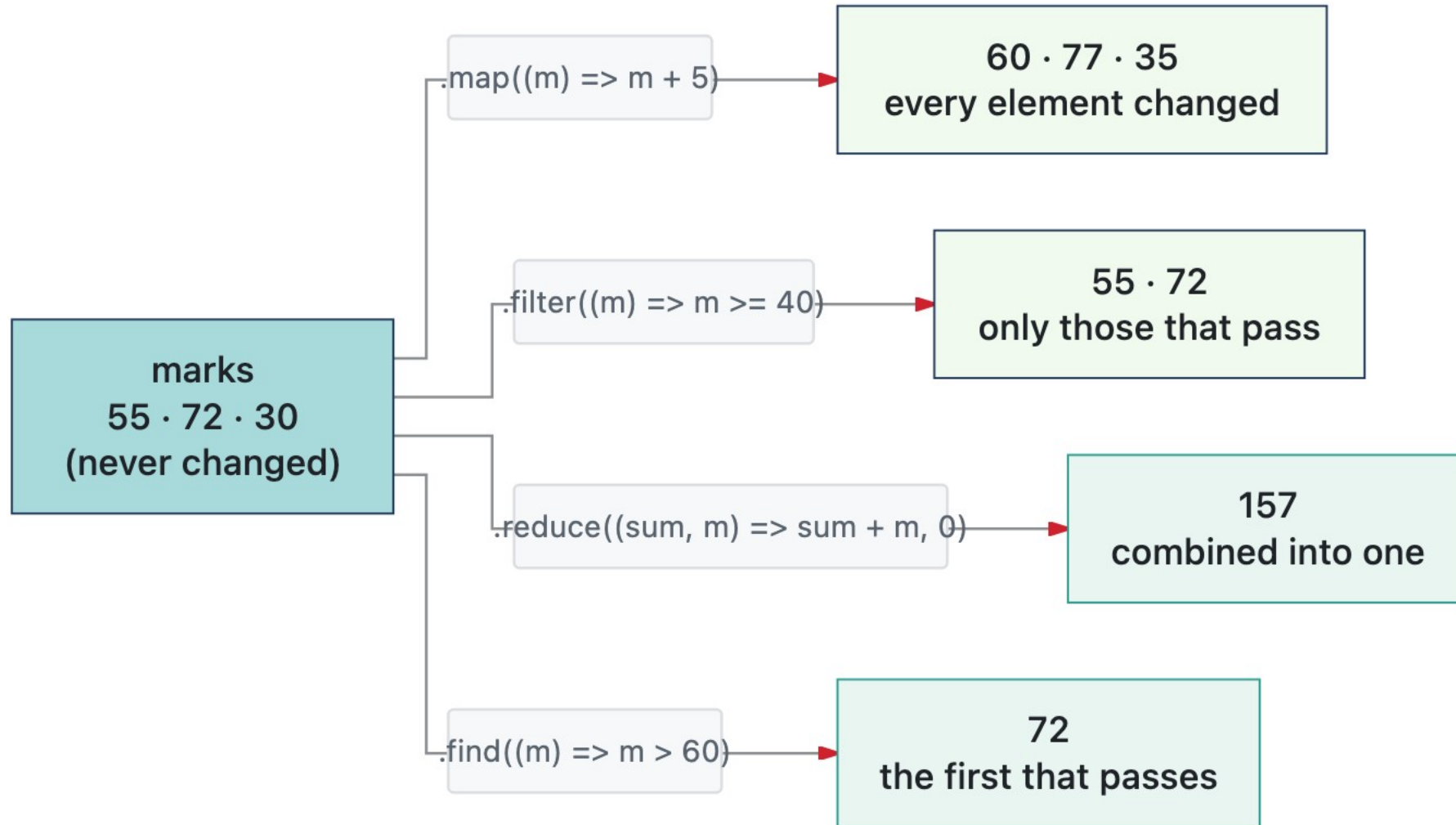
"if `resetButton` is null, do nothing; otherwise call it"

The `this` trap

```
resetButton?.addEventListener("click", home.reset); // wrong!  
resetButton?.addEventListener("click", () => home.reset()); // right
```

- no type error, no lint warning, tests pass - **and nothing happens**
- `home.reset` on its own is cut off from `home`
- the browser calls it with `this` = the button
- **never pass a method on its own - wrap it**

Project 3: working through arrays



map, filter, find

```
export type Student = { name: string; mark: number };

export const marksOf = (students: Student[]): number[] =>
  students.map((student) => student.mark);

export const passed = (students: Student[]): Student[] =>
  students.filter((student) => student.mark >= PASS_MARK);

export const findStudent = (
  students: Student[],
  name: string,
): Student | undefined => students.find((s) => s.name === name);
```

type names a type · find may give undefined

reduce

```
const total = marks.reduce((sum, mark) => sum + mark, 0);
```

step	sum	mark	result
1	0	50	50
2	50	60	110
3	110	70	180

Average, test first

```
Deno.test("the average of no marks is 0, not NaN", () => {  
  assertEquals(average([]), 0);  
});
```

- without a check: `0 / 0` is `NaN` → red
- add `if (marks.length === 0) return 0;` → green
- the test found the bug **before** a user did

Sorting

```
export const byMark = (students: Student[]): Student[] =>
  students.toSorted((a, b) => b.mark - a.mark);

export const byName = (students: Student[]): Student[] =>
  students.toSorted((a, b) => a.name.localeCompare(b.name));
```

- a **comparator**, as in Java: negative / zero / positive
- `toSorted` - a copy; `sort` changes the original

Chaining

```
rows.innerHTML = shown
  .map((s) => `|<td>${s.name}</td><td>${s.mark}</td></tr>`)
  .join("");
|  |

```

- each method gives a new array → call the next one on it
- `map` to HTML strings, `join` into one string
- (the real code adds a grade column too)

Project 4: functions in data

```
export type ConvertFunction = (value: number) => number;

export type Converter = {
  name: string;
  from: string;
  to: string;
  convert: ConvertFunction;
};

export const CONVERTERS: Converter[] = [
  {
    name: "Kilometres to miles", from: "km", to: "miles",
    convert: (km) => km * 0.621371,
  },
  // ...
];
```

(km) needs no type - TypeScript knows from Converter

Choosing a function

```
const converter = findConverter(choice.value);
const converted = convertAll(EXAMPLES, converter.convert);

export const convertAll = (
  values: number[],
  convert: ConvertFunction,
): number[] => values.map(convert);

Deno.test("convertAll applies the function to every value", () => {
  assertEquals(convertAll([1, 2, 3], (x) => x * 10), [10, 20, 30]);
});
```

New conversion = one new object. The idea behind **Strategy** (Book 3)

Summary

- `(params) => body` - expression body returns; block body needs `return`
- arrows go anywhere a value goes: listeners, `map`, objects
- closures remember variables
- `assertAlmostEquals` for decimals; `?.` for "if not null"
- **wrap methods:** `() => obj.method()`
- `map`, `filter`, `reduce`, `find`, `toSorted` - originals untouched
- `type` names; `(v: number) => number` is a function type

Challenges

1. **Kelvin** - `toKelvin`, test first
2. **Free throw and undo** - `undo()`, with `push`, `pop`, `reduce`
3. **Highest and lowest** - `bottomStudent`
4. **Grade counts** - `countGrade`, `map` + `join`
5. **New conversions, no new code** - and a better test
6. **Search as you type** - `searchByName`